LeetCode Practice

5 Hard: BFS

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

5
HARD

Patterns (in order)

1. BFS (5)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

5 questions | tap a title to jump to it

Hard questions (5) Go >

BFS (5) Go >

H 1. Word Ladder LC 127

H 2. Shortest Distance from All Buildings LC 317

H 3. Sliding Puzzle LC 773

H 4. Bus Routes LC 815

H 5. Shortest Path in a Grid with Obstacles Elimination LC 1293

Hard questions (5)

HARD | TOPIC 1 OF 1

BFS

5 Hard questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Word Ladder LC 127
2. Shortest Distance from All Buildings LC 317
3. Sliding Puzzle LC 773
4. Bus Routes LC 815
5. Shortest Path in a Grid with Obstacles Elimination LC 1293

HARD | BFS | LeetCode #127 | 1 of 5 | 1/5

Word Ladder

QUESTION

A transformation sequence from word beginWord to word endWord using a dictionary wordList is a sequence of words beginWord -> s_1 -> s_2 -> ... -> s_k such that:

CONSTRAINTS

- Every adjacent pair of words differs by a single letter.
- Every s_i for 1 <= i <= k is in wordList. Note that beginWord does not need to be in wordList.
- s_k == endWord

Given two words, beginWord and endWord, and a dictionary wordList, return the number of words in the shortest transformation sequence from beginWord to endWord, or 0 if no such sequence exists.

Example 1

Input	`beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log","cog"]`
Output	`5`
Explanation	*One shortest transformation sequence is "hit" -> "hot" -> "dot" -> "dog" -> cog", which is 5 words long.*

Example 2

Input	`beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log"]`
Output	`0`
Explanation	*The endWord "cog" is not in wordList, therefore there is no valid transformation sequence.*

- 1 <= beginWord.length <= 10
- endWord.length == beginWord.length
- 1 <= wordList.length <= 5000
- wordList[i].length == beginWord.length
- beginWord, endWord, and wordList[i] consist of lowercase English letters.
- beginWord != endWord
- All the words in wordList are unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Shortest word transformation from beginWord to endWord, changing one letter at a time.

Each intermediate word must be in wordList. Return total words in shortest path, or 0. Right?"

Can the string be empty? Are all characters ASCII or could they be Unicode?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
depth-first search over every simple path from
    beginWord
best = min(best, path)
```

TIME	SPACE
O(N!)	O(N)

WHY IT IS SLOW

The number of simple paths is exponential. Breadth-first search reaches each word at most once, in order of distance: O(N * L^2) (L = word length).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (BFS with pattern matching)

*"BFS on word transformations — already O(N*L*26). Walk-through of how BFS finds min steps:"*

Build a set from wordList for O(1) lookup.

Queue: [(beginWord, 1)] — (current_word, steps_so_far).

For each word popped:

For each position i (0..L-1): try replacing with each of 26 letters.

`candidate = word[:i] + letter + word[i+1:]`

If candidate == endWord: return steps+1 immediately.

If candidate in word_set and not visited: enqueue (candidate, steps+1). Remove from set.

WHY BFS gives minimum steps: BFS explores words at distance 1, then 2, then 3...

First time we reach endWord = fewest transformations needed.

WHY remove from set instead of a visited set: saves memory and achieves the same effect

(each word can only be visited once on the shortest path).

Return 0 if queue exhausted without reaching endWord.

Time: O(N*L*26). Space: O(N*L).

PHASE 4 CLEAN CODE

```
class _127_WordLadder:
    def ladderLength(self, beginWord: str, endWord: str, w
ordList: List[str]) -> int:
        word_set = set(wordList)
        if endWord not in word_set:
            return 0
        queue   = deque([(beginWord, 1)])
        visited = {beginWord}
        while queue:
            word, steps = queue.popleft()
            for i in range(len(word)):
                for c in 'abcdefghijklmnopqrstuvwxyz':
                    next_word = word[:i] + c + word[i+1:]
                    if next_word == endWord:
                        return steps + 1
                    if next_word in word_set and next_word
 not in visited:
```

```
                    visited.add(next_word)
                    queue.append((next_word, steps + 1
))
        return 0
```

PHASE 5 TEST & DEBUG

"Let me trace this BFS level-by-level using the queue state."

beginWord='hit', endWord='cog', wordList=['hot','dot','dog','lot','log','cog']:

word_set={'hot','dot','dog','lot','log','cog'}. 'cog' in set ok.

BFS: queue=[('hit',1)]. visited={'hit'}.

'hit': h->h: 'ait'? No. 'bit'? No. 'hot'! -> enqueue ('hot',2).

i->_: 'hat'? No. 'hut'? No. None in set.

t->_: 'hia'? No. etc.

'hot': h->_: 'aot'..'dot'! (2), 'lot'! (2). -> enqueue ('dot',3),('lot',3).

'dot': d->_: 'aot'..'lot' visited. o->_: 'dat'..'dog'! -> ('dog',4).

'dog': d->_:'aog'..'log'! ->('log',5). o->_:'dag'..'cog'!'cog'==endWord -> return 5 ok

WHY BFS: the queue processes nodes in increasing distance/level, so the first time we reach a node gives the shortest path for that level.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n * L * 26)	**O(n*L)**
= O(n*L)	Standard BFS shortest path

Bidirectional BFS: search simultaneously from beginWord and endWord — roughly halves depth.

Given more time I'd ask: return all shortest paths?

That's Word Ladder II (#126) — BFS to build levels, then DFS to backtrack paths.

HARD | BFS | LeetCode #317 | 2 of 5 | 2/5

Shortest Distance from All Buildings

QUESTION

You are given an m x n grid grid of values 0, 1, or 2, where:

CONSTRAINTS

- each 0 marks an empty land that you can pass by freely,
- each 1 marks a building that you cannot pass through, and
- each 2 marks an obstacle that you cannot pass through.

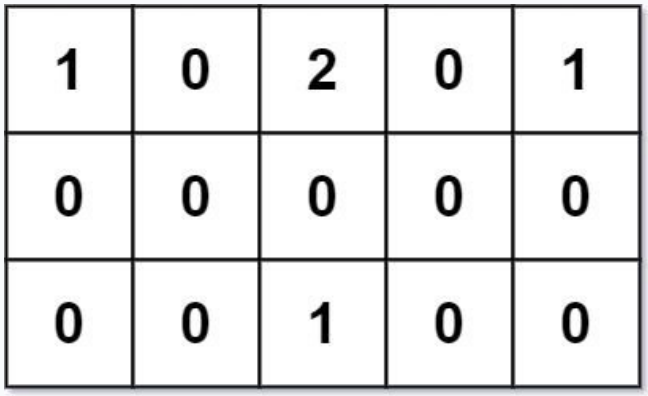
You want to build a house on an empty land that reaches all buildings in the shortest total travel distance. You can only move up, down, left, and right.

Return the shortest travel distance for such a house. If it is not possible to build such a house according to the above rules, return -1.

The total travel distance is the sum of the distances between the houses of the friends and the meeting point.

Example 1

Input	`grid = [[1,0,2,0,1],[0,0,0,0,0],[0,0,1,0,0]]`
Output	`7`
Explanation	*Given three buildings at (0,0), (0,4), (2,2), and an obstacle at (0,2). The point (1,2) is an ideal empty land to build a house, as the total travel distance of 3+3+1=7 is minimal. So return 7.*



Example 2

Input	`grid = [[1,0]]`
Output	`1`

Example 3

Input	`grid = [[1]]`
Output	`-1`

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 50

QUESTION

- grid[i][j] is either 0, 1, or 2.
- There will be at least one building in the grid.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Find empty land (0) minimising total BFS distance to all buildings (1).

Obstacles (2) can't be passed. Return -1 if no valid land. Right?"

Are inputs always non-negative integers within 32-bit range?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for cell in cells:
    run a BFS to every building and add up the
        distances
    keep the smallest total where all buildings
        were reached
```

TIME	SPACE
O((m * n)^2)	O(m * n)

WHY IT IS SLOW

A BFS per empty cell. Run a BFS from each building once, adding its distances into a shared grid and counting how many buildings reached each cell: O(B * m * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (BFS from each building, accumulate distances)

*"BFS from each building, accumulate distance + reach count — already O(k*mn). Walk-through:"*

For each building (grid[r][c]==1):

BFS from that building to all reachable empty cells.

For each empty cell (nr,nc) reached at distance d:

```
total_dist[nr][nc] += d <- add distance to running
  total
reach[nr][nc] += 1 <- increment how many buildings can
  reach this cell
```

After all BFS runs:

Answer = min(total_dist[r][c]) for cells where reach[r][c] == total_buildings.

WHY reach check: only cells reachable from ALL buildings are valid candidates.

Cells blocked from some buildings would give incorrect (incomplete) distance sums.

If no such cell exists -> return -1.

Optimization: if a BFS can't reach all empty cells (wall blocking), prune early.

PHASE 3 OPTIMIZE (BFS from each building, accumulate distances)

Time: O(k*mn) where k = buildings. Space: O(mn)."

Time: O(n) — optimal single-pass solution. Linear time is best achievable.

PHASE 4 CLEAN CODE

```
class _317_ShortestDistanceBuildings:
    def shortestDistance(self, grid: List[List[int]]) -> i
nt:
        m, n      = len(grid), len(grid[0])
        dist      = [[0] * n for _ in range(m)]
        reach     = [[0] * n for _ in range(m)]
        buildings = sum(grid[r][c] == 1 for r in range(m)
for c in range(n))

        for r in range(m):
            for c in range(n):
                if grid[r][c] != 1: continue
                queue   = deque([(r, c, 0)])
                visited = {(r, c)}
                while queue:
                    cr, cc, d = queue.popleft()
                    for dr, dc in [(-1,0),(1,0),(0,-1),(0,
1)]:
                        nr, nc = cr + dr, cc + dc
                        if 0 <= nr < m and 0 <= nc < n and
 grid[nr][nc] == 0 and (nr,nc) not in visited:
                            visited.add((nr, nc))
                            dist[nr][nc]  += d + 1
                            reach[nr][nc] += 1
                            queue.append((nr, nc, d + 1))

        return min(
            (dist[r][c] for r in range(m) for c in range(n
) if grid[r][c] == 0 and reach[r][c] == buildings),
            default=-1
        )
```

PHASE 5 TEST & DEBUG

"Let me trace shortestDistanceAllBuildings on [[1,0,2,0,1],[0,0,0,0,0],[0,0,1,0,0]]."

Multi-source BFS from each building (cell=1):

From building (0,0): BFS fills dist_from[0][0]. From (0,4): fills dist_from[0][4]. From (2,2): fills dist_from[2][2].

For each empty cell (0): total_dist = sum across all 3 BFS distances. Also need all buildings reachable.

Cell (1,2): dist from (0,0)=3, from (0,4)=3, from (2,2)=1. total=7. reachable_count=3 ok

Cell (0,2)=2 (obstacle). Skip.

Find min total_dist among cells where reachable_count==num_buildings.

WHY BFS per building: shortest path in unweighted grid = BFS levels. Obstacles block BFS.

WHY BFS details: this uses a queue; snapshots/levels keep distances correct and avoid mixing levels.

BFS from each building: queue=[(r,c,0)]. visited={} per BFS run. Mark reachable 0-cells. Sum distances across all BFS runs. Cells not visited by all buildings -> unreachable.

PHASE 5 TEST & DEBUG

Graph BFS: dist[cell]=sum of distances from all buildings. component: cells reachable from ALL buildings. Unreachable from any -> excluded. Return min dist.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(buildings * m * n)	**O(m*n)** BFS from each building accumulates into dist[]

reach[] ensures we only pick land reachable from ALL buildings.

Given more time I'd ask: what if buildings can move? Re-run BFS on each query.

HARD | BFS | LeetCode #773 | 3 of 5 | 3/5

Sliding Puzzle

QUESTION

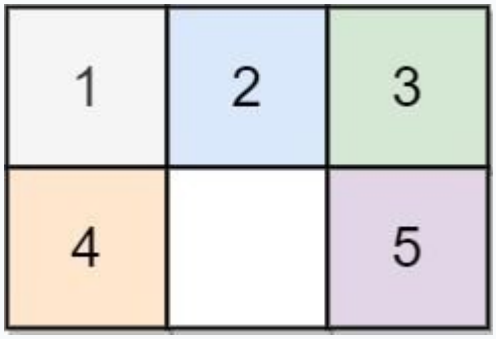
On an 2 x 3 board, there are five tiles labeled from 1 to 5, and an empty square represented by 0. A move consists of choosing 0 and a 4-directionally adjacent number and swapping it.

The state of the board is solved if and only if the board is [[1,2,3],[4,5,0]].

Given the puzzle board board, return the least number of moves required so that the state of the board is solved. If it is impossible for the state of the board to be solved, return -1.

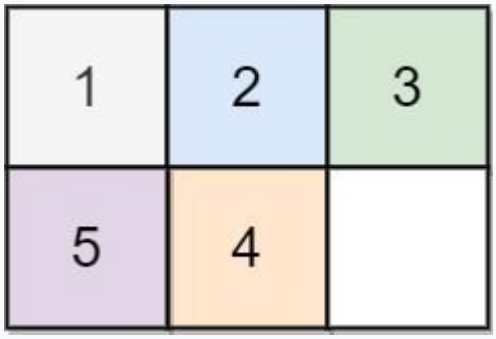
Example 1

Input	`board = [[1,2,3],[4,0,5]]`
Output	`1`
Explanation	*Swap the 0 and the 5 in one move.*



Example 2

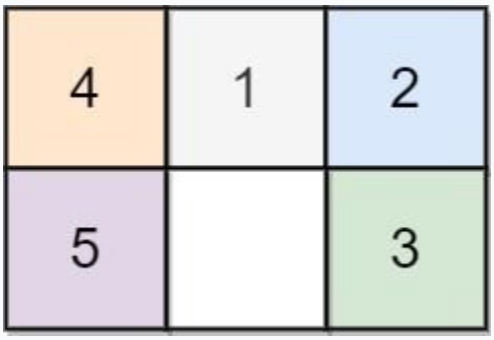
Input	`board = [[1,2,3],[5,4,0]]`
Output	`-1`
Explanation	*No number of moves will make the board solved.*



QUESTION

Example 3

Input	`board = [[4,1,2],[5,0,3]]`
Output	`5`
Explanation	*5 is the smallest number of moves that solves the board.* *An example path:* *After move 0: [[4,1,2],[5,0,3]]* *After move 1: [[4,1,2],[0,5,3]]* *After move 2: [[0,1,2],[4,5,3]]* *After move 3: [[1,0,2],[4,5,3]]* *After move 4: [[1,2,0],[4,5,3]]* *After move 5: [[1,2,3],[4,5,0]]*



CONSTRAINTS

- board.length == 2
- board[i].length == 3
- 0 <= board[i][j] <= 5
- Each value board[i][j] is unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

2×3 board. Swap 0 with adjacent tile. Minimum moves to reach [[1,2,3],[4,5,0]]. Right?

Return -1 if unsolvable."

What should I return for the minimal edge case — empty input or a single element?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def dfs(board, moves):
    slide the 0 in every legal direction and
        recurse without remembering visited
        boards
    best = min(best, moves)
```

TIME	SPACE
exponential	O(depth)

WHY IT IS SLOW

Revisits boards again and again. BFS from the start board with a visited set explores each of the 360 reachable states once: O(6!).

PHASE 2 BRUTE FORCE

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (BFS on serialised board state)

*"BFS on serialized board states — O(6! * 6) since at most 720 states. Walk-through:"*

Represent board as a 6-char string (left->right, top->bottom). e.g. [[1,2,3],[4,0,5]] = '123405'.

Goal state: '123450'.

Precompute adjacency: for flat index i (position of 0), which positions can it swap with?

Original 2×3 grid adjacency: position 0{1,3}, 1{0,2,4}, 2{1,5}, 3{4,0}, 4{3,5,1}, 5{4,2}.

BFS from start state. For each state popped:

Find position of '0'. Try swapping with each adjacent position.

Generate new state strings. If not visited: enqueue.

First time we see '123450' -> return steps. If queue empties -> return -1.

WHY serialization: BFS state must be hashable; string/tuple of board works.

Time: O(6! * 6). Space: O(6!).

PHASE 4 CLEAN CODE

```
class _773_SlidingPuzzle:
    def slidingPuzzle(self, board: List[List[int]]) -> int
:
        GOAL   = '123450'
        NEIGHBORS = {0:[1,3], 1:[0,2,4], 2:[1,5], 3:[0,4],
 4:[1,3,5], 5:[2,4]}
        start  = ''.join(str(board[r][c]) for r in range(2
) for c in range(3))
        queue  = deque([(start, 0)])
        visited = {start}

        while queue:
            state, moves = queue.popleft()
            if state == GOAL:
                return moves
            z = state.index('0')
            for nb in NEIGHBORS[z]:
                new_state = list(state)
                new_state[z], new_state[nb] = new_state[nb
], new_state[z]
                ns = ''.join(new_state)
                if ns not in visited:
                    visited.add(ns)
                    queue.append((ns, moves + 1))
        return -1
```

PHASE 5 TEST & DEBUG

"Let me trace Sliding Puzzle with the BFS queue and neighbor swaps."

Start grid [[1,2,3],[4,0,5]] -> state = '123405'.

Goal state = '123450'.

Locate 0 in '123405': index 4 (row 1, col 1). In a 2x3 board, index 4 can swap with left=3 and right=5.

Level 0 (buses/steps=0): queue=['123405'] ; visited={'123405'}.

PHASE 5 TEST & DEBUG

Pop '123405':

swap 0 with left (index 3): '123405' -> '123045'. (matrix [[1,2,3],[0,4,5]])

swap 0 with right (index 5): '123405' -> '123450'. (matrix [[1,2,3],[4,5,0]])

Level 1 (steps=1): we discover '123450' immediately, so we return steps=1 ok

WHY BFS: each queue layer represents all states reachable in the same number of swaps, so the first time we hit the goal is the shortest path.

Base case: if start==goal we would return 0 before expanding.

PHASE 6 COMPLEXITY & FOLLOW-UP

States = 6! / 2 = 360 reachable states (half of 6! are unsolvable). BFS on 360 states.

The hardcoded NEIGHBORS map is the key — it captures the 2D adjacency in 1D string indexing.

Given more time I'd ask: generalise to n×m board?

Same BFS approach — NEIGHBORS changes, state space = (n*m)! / 2."

Time: O(n) — one-pass BFS/DFS sliding tokens. Space: O(n) — queue/stack for pending states.

HARD | BFS | LeetCode #815 | 4 of 5 | 4/5

Bus Routes

QUESTION

You are given an array routes representing bus routes where routes[i] is a bus route that the ith bus repeats forever.

CONSTRAINTS

- For example, if routes[0] = [1, 5, 7], this means that the 0th bus travels in the sequence 1 -> 5 -> 7 -> 1 -> 5 -> 7 -> 1 -> ... forever.

You will start at the bus stop source (You are not on any bus initially), and you want to go to the bus stop target. You can travel between bus stops by buses only.

Return the least number of buses you must take to travel from source to target. Return -1 if it is not possible.

Example 1	
Input	`routes = [[1,2,7],[3,6,7]], source = 1, target = 6`
Output	`2`
Explanation	*The best strategy is take the first bus to the bus stop 7, then take the second bus to the bus stop 6.*

Example 2	
Input	`routes = [[7,12],[4,5,15],[6],[15,19],[9,12,13]], source = 15, target = 12`
Output	`-1`

- 1 <= routes.length <= 500.
- 1 <= routes[i].length <= 10^5
- All the values of routes[i] are unique.
- sum(routes[i].length) <= 10^5
- 0 <= routes[i][j] < 10^6
- 0 <= source, target < 10^6

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Minimum number of BUSES (not stops) to travel from source to target. Right?

Each bus covers its entire route — stepping onto a bus travels all its stops."

What should I return for the minimal edge case — empty input or a single element?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for set in bus_routes:
    check whether they connect the source to the
        target
    best = min(best, set)
```

TIME	SPACE
O(2^R)	O(R)

WHY IT IS SLOW

Tries every subset of routes. Map each stop to the routes through it and BFS over routes (a bus is a node, a shared stop is an edge): O(total stops).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (BFS on routes, not stops)

"BFS on ROUTES (not stops) — already O(R^2 + S) where R=routes, S=stops. Walk-through:"

Build stop -> list of route indices map.

BFS state = a route index (not a bus stop).

Initialize: find all routes containing source stop -> enqueue them (buses=1).

BFS level = 1 bus transfer:

Pop a route. Visit ALL stops on this route.

For each stop: if it's the target -> return buses count.

For each route that shares this stop (from stop->routes map):

If not visited: enqueue that route (one more bus transfer needed).

WHY BFS on ROUTES not stops: stops can be reached via many routes; modeling on routes

avoids re-visiting the same stop through different buses at the same BFS depth.

Time: O(total stops across all routes). Space: O(same)."

Time: O(n) — optimal single-pass solution. Linear time is best achievable.

PHASE 4 CLEAN CODE

```python
class _815_BusRoutes:
    def numBusesToDestination(self, routes: List[List[int]], source: int, target: int) -> int:
        if source == target:
            return 0
        stop_to_routes: defaultdict = defaultdict(set)
        for i, route in enumerate(routes):
            for stop in route:
                stop_to_routes[stop].add(i)

        visited_stops  = {source}
        visited_routes = set()
        queue = deque([(source, 0)])          # (stop, buses_taken)

        while queue:
```

```
            stop, buses = queue.popleft()
            for route_id in stop_to_routes[stop]:
                if route_id in visited_routes: continue
                visited_routes.add(route_id)
                for next_stop in routes[route_id]:
                    if next_stop == target:
                        return buses + 1
                    if next_stop not in visited_stops:
                        visited_stops.add(next_stop)
                        queue.append((next_stop, buses + 1
))
        return -1
```

PHASE 5 TEST & DEBUG

"Let me trace numBusesToDestination([[1,2,5],[2,6],[1,8]], S=1, T=6)."

BFS on buses (not stops): from S=1, what buses contain stop 1?

Build stop->buses map: 1->{0,2}, 2->{0,1}, 5->{0}, 6->{1}, 8->{2}

BFS queue: start with buses containing S=1: buses {0,2}

Level 1 (1 bus): process bus 0: stops 1,2,5 -> add new buses for stop 2: bus1 -> add to next level

process bus 2: stops 1,8 -> no T=6 yet

Level 2 (2 buses): process bus 1: stops 2,6 -> found T=6! Return 2 ok

WHY BFS on buses: each bus route = one transfer; minimize transfers not stops.

BFS: queue=[(source_stop,0)]. visited_stops={source_stop}. visited_routes={} marks expanded routes. Pop stop: for each route through it: enqueue all stops on that route not yet visited.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(sum of route lengths)	**O(same)** BFS on routes, not stops — the key insight

Visiting stops prevents re-enqueuing; visiting routes prevents re-processing entire routes.

Given more time I'd ask: find all paths using at most k buses?

Keep track of bus count per path — BFS naturally handles this.

HARD | BFS | LeetCode #1293 | 5 of 5 | 5/5

Shortest Path in a Grid with Obstacles Elimination

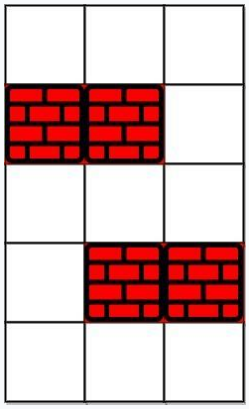
QUESTION

You are given an m x n integer matrix grid where each cell is either 0 (empty) or 1 (obstacle). You can move up, down, left, or right from and to an empty cell in one step.

Return the minimum number of steps to walk from the upper left corner (0, 0) to the lower right corner (m - 1, n - 1) given that you can eliminate at most k obstacles. If it is not possible to find such walk return -1.

Example 1

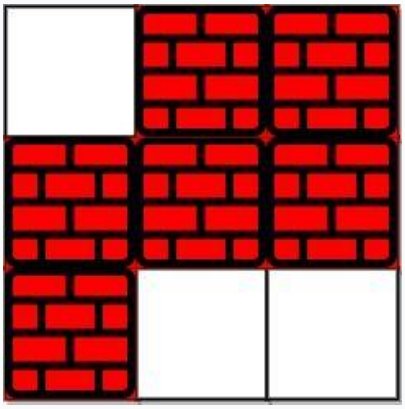
Input	`grid = [[0,0,0],[1,1,0],[0,0,0], [0,1,1],[0,0,0]], k = 1`
Output	**6**
Explanation	*The shortest path without eliminating any obstacle is 10.* *The shortest path with one obstacle elimination at position (3,2) is 6. Such path is (0,0) -> (0,1) -> (0,2) -> (1,2) -> (2,2) -> (3,2)* *-> (4,2).*



Example 2

Input	`grid = [[0,1,1],[1,1,1],[1,0,0]], k = 1`
Output	**-1**
Explanation	*We need to eliminate at least two obstacles to find such a walk.*

QUESTION



CONSTRAINTS

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 40
- 1 <= k <= m * n
- grid[i][j] is either 0 or 1.
- grid[0][0] == grid[m - 1][n - 1] == 0

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find shortest path from (0,0) to (m-1,n-1) in a binary grid; can eliminate at most k obstacles (1s).

Return -1 if impossible. Right?"

"A few quick questions:

0=free, 1=obstacle. State = (row, col, remaining_eliminations)? Yes."

"Let me walk: grid=[[0,0,0],[1,1,0],[0,0,0],[0,1,1],[0,0,0]], k=1 -> 6 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def dfs(cell, removals left):
    move in the 4 directions, removing an
        obstacle when needed and allowed, without
        remembering states
    best = min(best, candidate)
```

TIME	SPACE
O(4^(m * n))	O(m * n)

WHY IT IS SLOW

Explores every path. BFS over states (row, column, removals left) with a visited table explores each state once: O(m * n * k).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"BFS with elimination tracking — already O(mnk). Walk-through:"

State = (row, col, remaining_eliminations). Start = (0, 0, k).

BFS expands states level by level (each level = 1 step).

For each state (r, c, rem) popped:

If (r,c) = bottom-right -> return steps.

For each of 4 directions (nr, nc):

If grid[nr][nc] == 0: enqueue (nr, nc, rem, steps+1) if not visited.

If grid[nr][nc] == 1 and rem > 0: enqueue (nr, nc, rem-1, steps+1) (use an elimination).

Visited key = (r, c, rem) — same cell with MORE eliminations remaining is a BETTER state.

WHY track remaining eliminations in state: same cell reached with different rem values

are different states (more eliminations = more options ahead -> not interchangeable).

Time: O(mnk). Space: O(mnk)."

Time: O(log n) — binary search on answer space.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import deque
class _1293_ShortestPathGridObstacles:
    def shortestPath(self, grid, k):
        m, n = len(grid), len(grid[0])
        if m == 1 and n == 1:
            return 0
        queue = deque([(0, 0, k, 0)])
        visited = {(0, 0, k)}
        while queue:
            r, c, remaining, dist = queue.popleft()
            for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
                nr, nc = r+dr, c+dc
                if not (0 <= nr < m and 0 <= nc < n):
                    continue
                new_k = remaining - grid[nr][nc]
                if new_k < 0:
                    continue
                if nr == m-1 and nc == n-1:
                    return dist + 1
                if (nr, nc, new_k) not in visited:
                    visited.add((nr, nc, new_k))
                    queue.append((nr, nc, new_k, dist + 1)
)
        return -1
```

PHASE 5 TEST & DEBUG

"Let me trace shortestPath on grid=[[0,0,0],[1,1,0],[0,0,0],[0,1,1],[0,0,0]], k=1."

BFS with state (row,col,remaining_eliminations)

Start: (0,0,k=1,dist=0). Target (4,2).

BFS explores level by level with state (r,c,k_remaining)

PHASE 5 TEST & DEBUG

Path: (0,0)->(1,0)[obstacle,k->0]->(2,0)->(3,0)->(4,0)->(4,1)->(4,2): dist=6 ok

WHY state includes k: same cell reachable with different remaining eliminations = different states.

Visited set = (r,c,k) triples to avoid revisiting.

State (r,c,k) ensures different k-remaining values = different valid paths. O(m*n*k) space.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(mnk)	**O(mnk)** visited set

Follow-up: Shortest Path in Binary Matrix (#1091) — k=0 special case.

Follow-up: if k is very large (>=m+n-2), Manhattan distance is the answer.